

Assignment No. 3

To develop a distributed system, to find the minimum number of N elements in an array by distributing N/n elements to n number of processors MPI or OpenMP. Demonstrate by displaying the intermediate minimum element calculated at different processors.

```
#include <stdio.h>
#include "mpi.h"

int main(int argc, char* argv[])
{
    int rank, size;
    int num[20];

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    for(int i=0;i<20;i++)
        num[i]=i+1;

    if(rank == 0)
    {
        int s[4];
        printf("Distribution at rank %d\n", rank);

        for(int i=1;i<4;i++)
            MPI_Send(&num[i*5], 5, MPI_INT, i, 1, MPI_COMM_WORLD);

        int local_min = num[0];

        for(int i=1;i<5;i++)
            if(num[i] < local_min)
                local_min = num[i];

        for(int i=1;i<4;i++)
            MPI_Recv(&s[i], 1, MPI_INT, i, 1, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

        printf("local min at rank %d is %d\n", rank, local_min);

        int final_min = local_min;

        for(int i=1;i<4;i++)
            if(s[i] < final_min)
                final_min = s[i];

        printf("final minimum = %d\n", final_min);
    }
    else
    {
        int k[5];
        MPI_Recv(k, 5, MPI_INT, 0, 1, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

        int local_min = k[0];

        for(int i=1;i<5;i++)
```

```
        if(k[i] < local_min)
            local_min = k[i];

        printf("local min at rank %d is %d\n", rank, local_min);

        MPI_Send(&local_min, 1, MPI_INT, 0, 1, MPI_COMM_WORLD);
    }

    MPI_Finalize();
    return 0;
}
```

Output

Distribution at rank 0

local min at rank 0 is 1

final minimum = 1

local min at rank 1 is 6

local min at rank 2 is 11

local min at rank 3 is 16